

Lecture 2. Kinematic and Dynamic Models

Kinematic and Dynamic Models

- Vehicles (this time)
- Manipulators (next time)

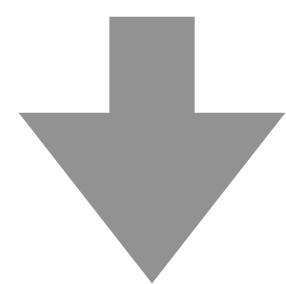
Remark

- In the context of control theory, all models describing the evolution of a system in time are dynamic models.
- In the context of robotics, there is a difference between Kinematics and Dynamics
 - Kinematics do not consider forces
 - Dynamics consider forces

Remark

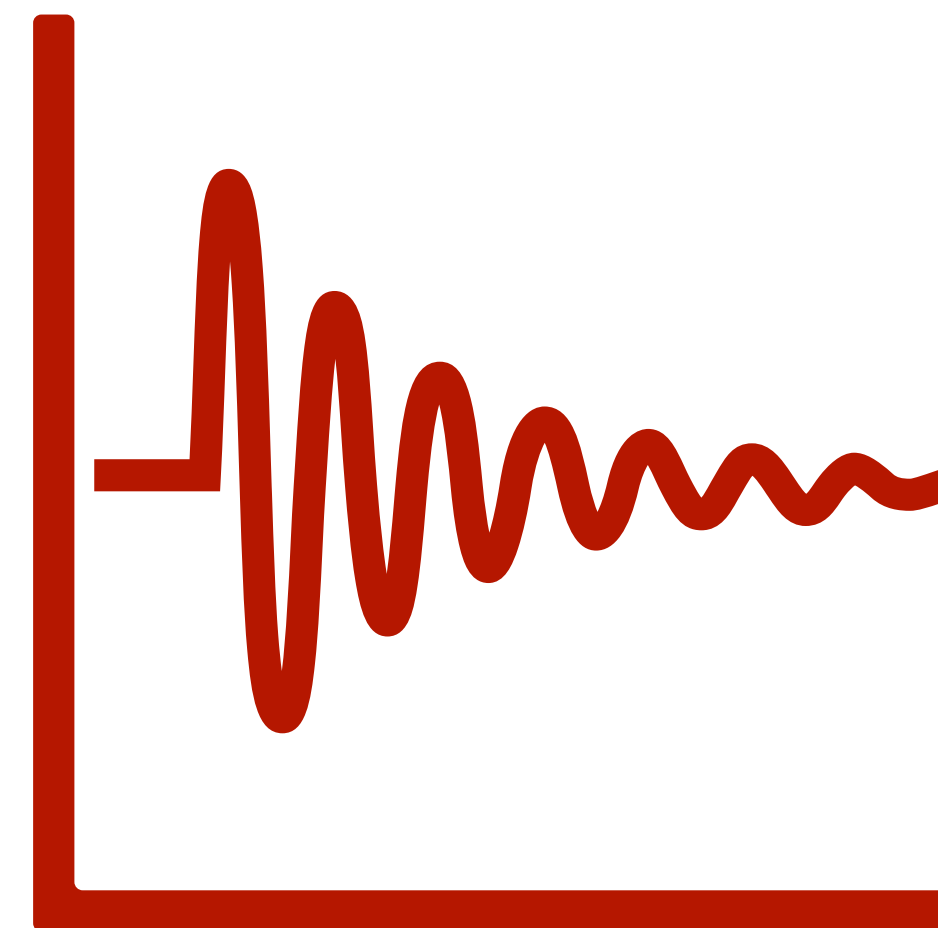
- Continuous time (CT) models vs Discrete time (DT) models
- We may turn CT models into DT models

$$\dot{x} = f(x, u)$$



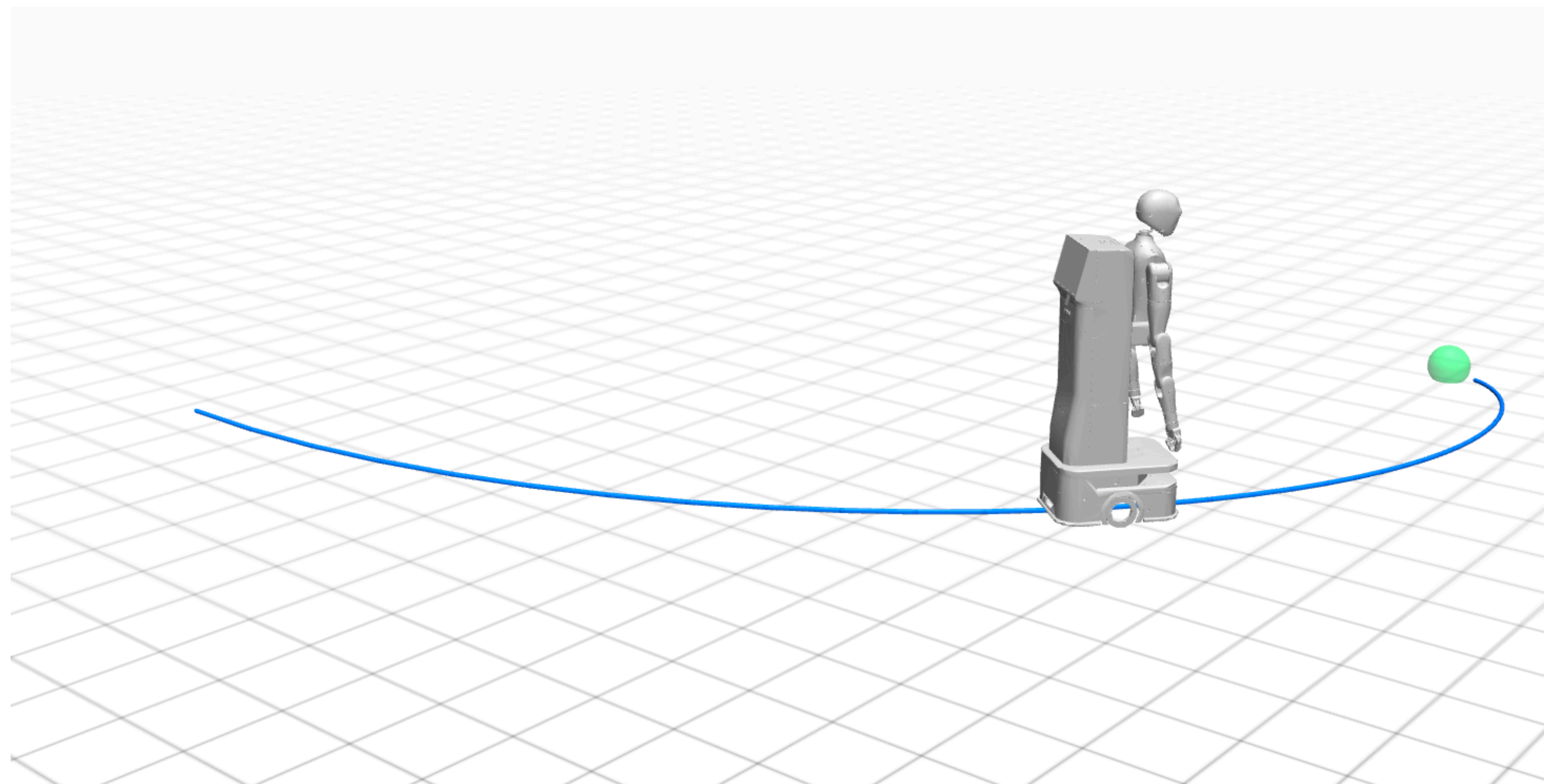
Euler Method

$$x_{k+1} = x_k + f(x_k, u_k)\Delta t$$



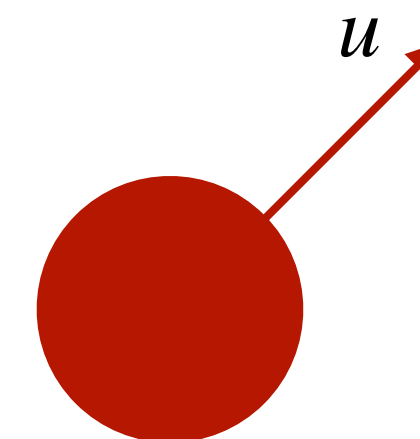
Vehicle Models

- Single integrator
- Double integrator
- Dubins car
- Unicycle - 3 state
- Unicycle - 4 state
- Dynamic Bicycle



Single Integrator

- State $x \in \mathbb{R}^2$: position
- Control $u \in \mathbb{R}^2$: velocity
- Dynamic equation
 - $\dot{x} = u$
 - $x_{k+1} = x_k + \Delta t u_k$



Single Integrator

Advantage

Simple

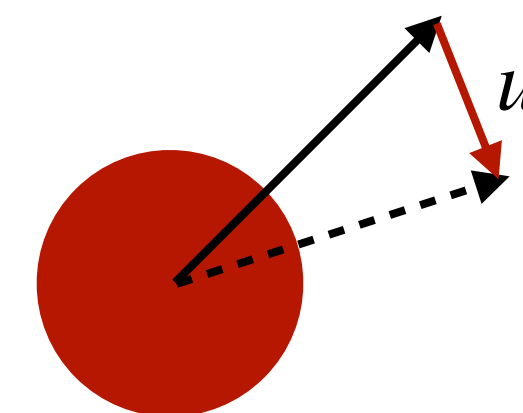
Disadvantage

Physically not achievable (inertia and constraints not considered)

- * Can you move a vehicle at $+\infty$ speed and then immediately move it at $-\infty$ speed?
- * Can you immediately change the heading of a vehicle?

Double Integrator

- State $x \in \mathbb{R}^4$: position and velocity
- Control $u \in \mathbb{R}^2$: acceleration
- Dynamic equation



- $\dot{x} = \begin{bmatrix} 0_2 & I_2 \\ 0_2 & 0_2 \end{bmatrix} x + \begin{bmatrix} 0_2 \\ I_2 \end{bmatrix} u$

- $x_{k+1} = \begin{bmatrix} I_2 & \Delta t I_2 \\ 0_2 & I_2 \end{bmatrix} x_k + \begin{bmatrix} 0_2 \\ \Delta t I_2 \end{bmatrix} u_k$

Inertia

Zero-order-hold (ZOH)

$$x_{k+1} = \begin{bmatrix} I_2 & \Delta t I_2 \\ 0_2 & I_2 \end{bmatrix} x_k + \begin{bmatrix} \frac{\Delta t^2}{2} I_2 \\ \Delta t I_2 \end{bmatrix} u_k$$

Double Integrator

Advantage

Relatively Simple

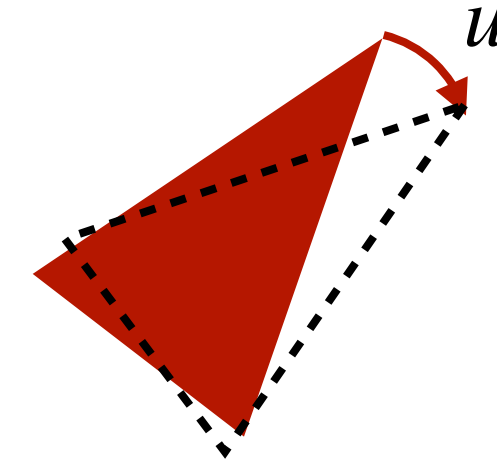
Inertia Considered

Disadvantage

Vehicle heading is not explicitly modeled

Dubins Car

- State $x \in \mathbb{R}^3$: position and heading $x = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \end{bmatrix}$
- Control $u \in \mathbb{R}$: turning rate
- Velocity $v \in \mathbb{R}$ is constant
- Dynamic equation



$$\dot{x} = \begin{bmatrix} v \cos x(3) \\ v \sin x(3) \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} u$$

$$x_{k+1} = x_k + \begin{bmatrix} v \cos x_k(3) \Delta t \\ v \sin x_k(3) \Delta t \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \Delta t \end{bmatrix} u_k$$

Dubins Car

Advantage

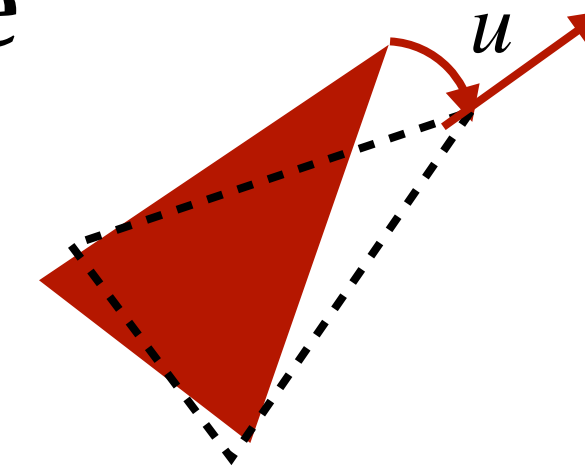
Heading considered

Disadvantage

Velocity does not change

Unicycle - 3 State

- State $x \in \mathbb{R}^3$: position and heading $x = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \end{bmatrix}$
- Control $u \in \mathbb{R}^2$: longitudinal velocity and turning rate
- Dynamic equation



$$\dot{x} = \begin{bmatrix} \cos x(3) & 0 \\ \sin x(3) & 0 \\ 0 & 1 \end{bmatrix} u$$

$$x_{k+1} = x_k + \begin{bmatrix} \cos x_k(3)\Delta t & 0 \\ \sin x_k(3)\Delta t & 0 \\ 0 & \Delta t \end{bmatrix} u_k$$

Unicycle 3 state

Advantage

Velocity changable

Heading considered

Disadvantage

Inertia not fully considered

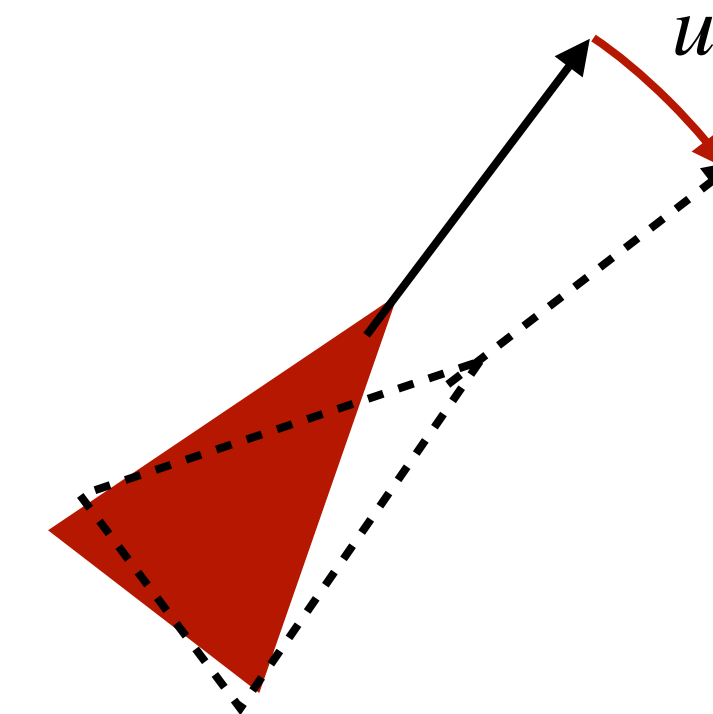
* Systems usually cannot tolerate big jumps in velocity

Unicycle - 4 State

- State $x \in \mathbb{R}^4$: position, longitudinal velocity, and heading
- Control $u \in \mathbb{R}^2$: longitudinal acceleration, and turning rate
- Dynamic equation

$$\dot{x} = \begin{bmatrix} x(3) \cos x(4) \\ x(3) \sin x(4) \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} u$$

$$x = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ x(4) \end{bmatrix}$$



$$x_{k+1} = x_k + \begin{bmatrix} x(3) \cos x(4) \Delta t \\ x(3) \sin x(4) \Delta t \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} u_k$$

Unicycle 4 state

Advantage

Inertia Considered

Heading considered

Disadvantage

More advanced dynamics not considered, e.g., drift

Bicycle Dynamic Model

- State $x \in \mathbb{R}^6$: position, longitudinal velocity, lateral velocity, yaw rate, yaw angle
 $x = [p_x \ p_y \ v_x \ v_y \ r \ \psi]^\top$
- Control $u \in \mathbb{R}^2$: longitudinal force, and front wheel steering angle
 $u = [F_x \ \delta_f]^\top$
- Dynamic equation

$$\dot{p}_x = v_x \cos \psi - v_y \sin \psi$$

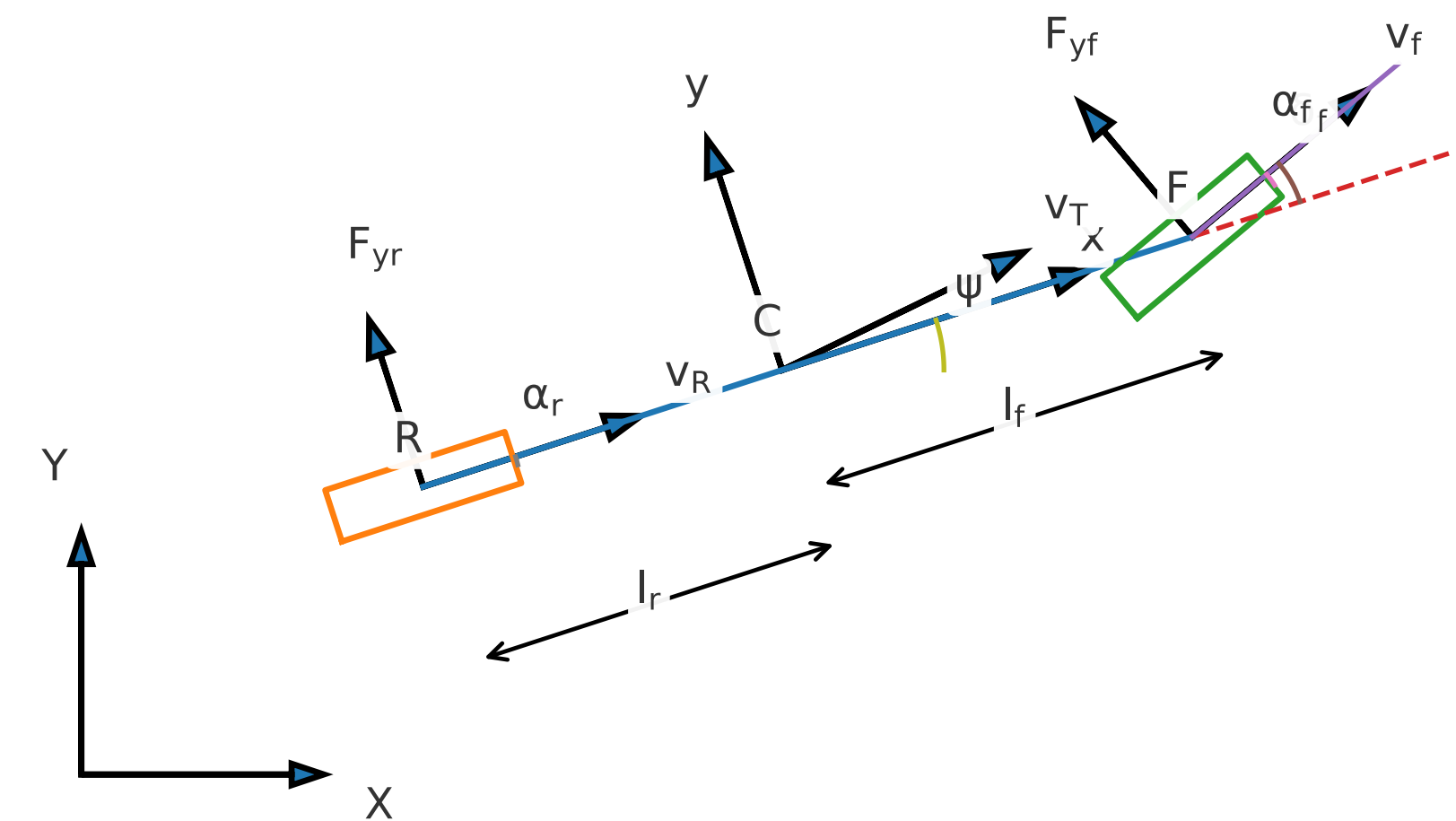
$$\dot{p}_y = v_x \sin \psi + v_y \cos \psi$$

$$m(\dot{v}_x - r v_y) = F_{xf} \cos \delta_f - F_{yf} \sin \delta_f + F_{xr}$$

$$m(\dot{v}_y + r v_x) = F_{xf} \sin \delta_f + F_{yf} \cos \delta_f + F_{yr}$$

$$I_z \dot{r} = l_f (F_{xf} \sin \delta_f + F_{yf} \cos \delta_f) - l_r F_{yr}$$

$$\dot{\psi} = r$$



$$F_{xf} = \beta F_x, \quad F_{xr} = (1 - \beta) F_x, \quad 0 \leq \beta \leq 1$$

$$F_{yf} = -C_f \left[\arctan \left(\frac{v_y + l_f r}{\max(v_x, \varepsilon)} \right) - \delta_f \right]$$

$$F_{yr} = -C_r \arctan \left(\frac{v_y - l_r r}{\max(v_x, \varepsilon)} \right)$$

Bicycle Model

Advantage

Inertia Considered

Geometry considered

Drift considered

Disadvantage

Highly nonlinear

Kinematic Vehicle Models

Model	State	Control	Continuous Time Dynamics	Discrete Time Dynamics
Single Integrator				
Double Integrator				
Dubins				
Unicycle3				
Unicycle4				
Dynamic Bicycle				

Remarks

- There are other more advanced models:
 - Four-wheel model: more like a passenger vehicle
- When do we use these models?
 - To control the ego vehicle
 - To predict the motion of other vehicles

Remarks

- What if the model is not accurate?
 - If we are controlling a real hardware system, there will always be **model mismatch**.
 - There are control techniques to compensate the model mismatch.*
 - The purpose to study these simple models is to derive control strategies that can be applied to more complex models.

Remarks

- How to choose among different models?
- We need to pick a model that is most suitable for the task.
 - For highway driving, it is more important to consider inertia than constraints. Then we can use double integrator.
 - For low-speed driving (e.g., in parking lots), it is more important to consider constraints than inertia. Then we can use unicycle (3 state).

Remarks

- Linear Models
 - $\dot{x} = Ax + Bu$
 - $x_{k+1} = Ax_k + Bu_k$
- Among all models we discussed, which of them are linear?

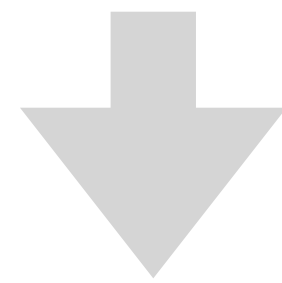
Remarks

- Although not all models are linear, one thing in common for all kinematic models (excluding the dynamic bicycle model):
 - The models are linear in control
 - $\dot{x} = f(x) + g(x)u$
 - $x_{k+1} = f(x_k) + g(x_k)u_k$
- These models are called **control-affine dynamics**

Model Linearization

$$x_{k+1} = f(x_k, u_k) \quad \text{Reference} \quad (\bar{x}_k, \bar{u}_k)$$

$$\delta x_k := x_k - \bar{x}_k, \quad \delta u_k := u_k - \bar{u}_k$$



$$\delta x_{k+1} \approx A_k \delta x_k + B_k \delta u_k + d_k$$

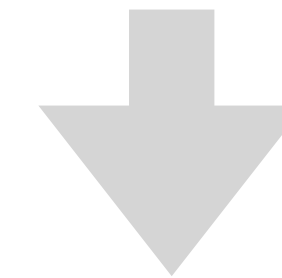
$$A_k = \left. \frac{\partial f}{\partial x} \right|_{(\bar{x}_k, \bar{u}_k)}, \quad B_k = \left. \frac{\partial f}{\partial u} \right|_{(\bar{x}_k, \bar{u}_k)}, \quad d_k = \underline{f(\bar{x}_k, \bar{u}_k) - \bar{x}_{k+1}}$$

Error in the reference trajectory

Example: Discrete Time Unicycle 3-State

$$x_k = [p_k^x; p_k^y; \theta_k] \quad u_k = [v_k; w_k]$$

$$x_{k+1} = x_k + \begin{bmatrix} \cos \theta_k \Delta t & 0 \\ \sin \theta_k \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} u_k$$



$$A_k = \begin{bmatrix} 1 & 0 & -\Delta t \bar{v}_k \sin \bar{\theta}_k \\ 0 & 1 & \Delta t \bar{v}_k \cos \bar{\theta}_k \\ 0 & 0 & 1 \end{bmatrix}$$

$$B_k = \begin{bmatrix} \Delta t \cos \bar{\theta}_k & 0 \\ \Delta t \sin \bar{\theta}_k & 0 \\ 0 & \Delta t \end{bmatrix}$$

Simulation

- Simulation
 - Matlab: standalone, control oriented
 - Python (New in 2026): integration with more robotics tools
 - Tutorial next lecture

<https://github.com/intelligent-control-lab/16-714>

Continuous Time to Discrete Time

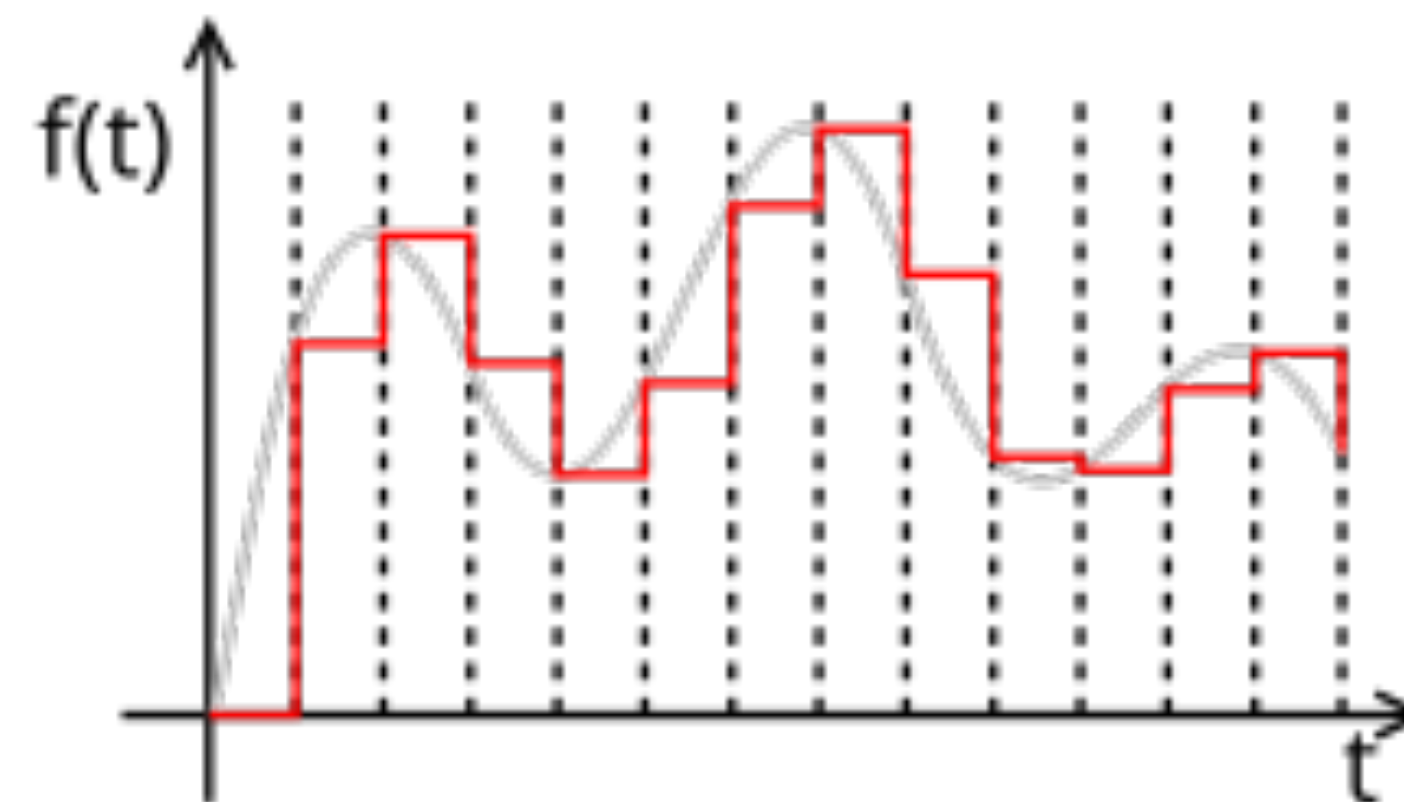
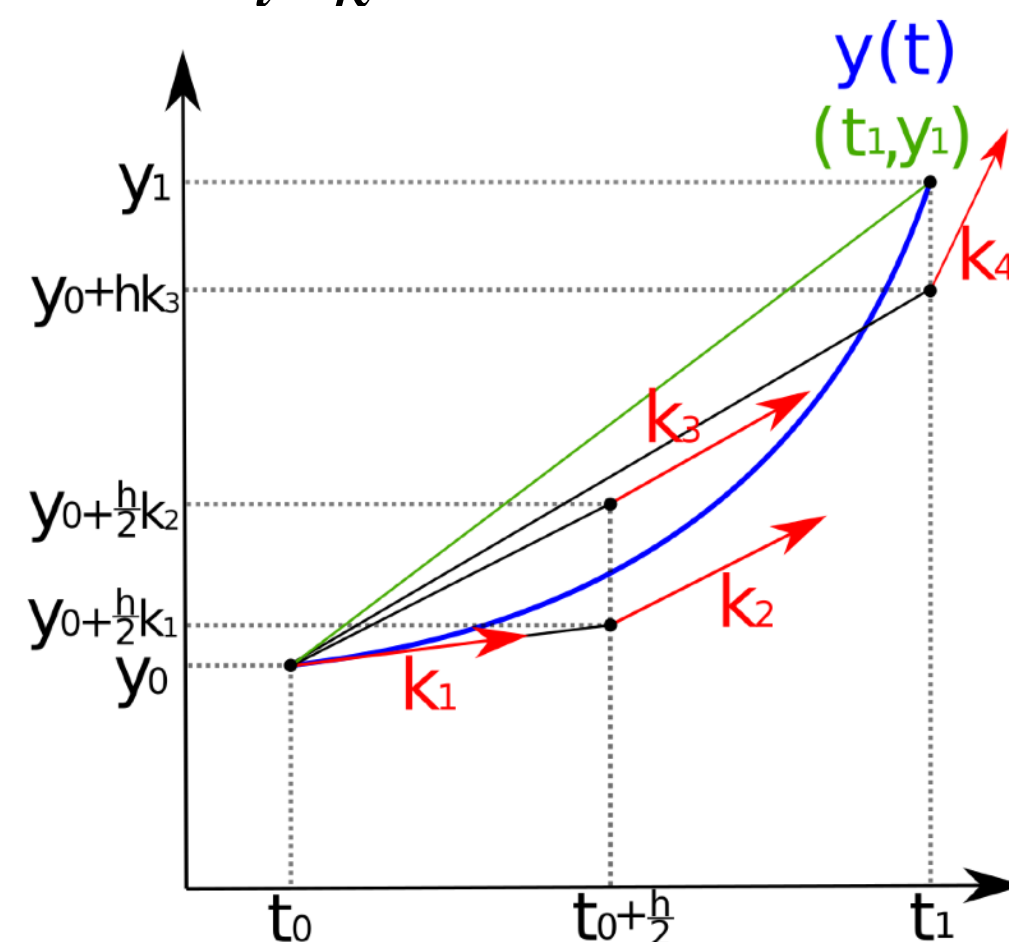
- Euler

- $x_{k+1} = x_k + f(x_k, u_k)\Delta t$

- ZOH

- $x_{k+1} = x_k + \int_{t=k}^{k+1} f(x_t, u_t)dt$

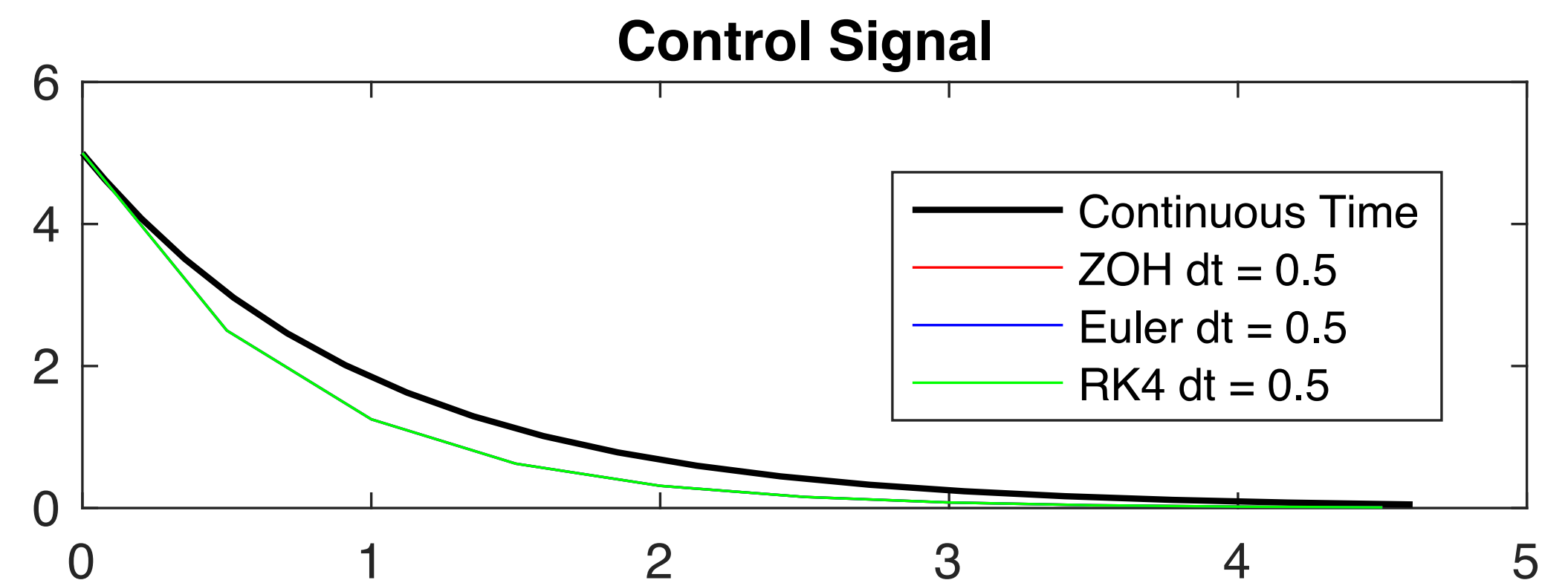
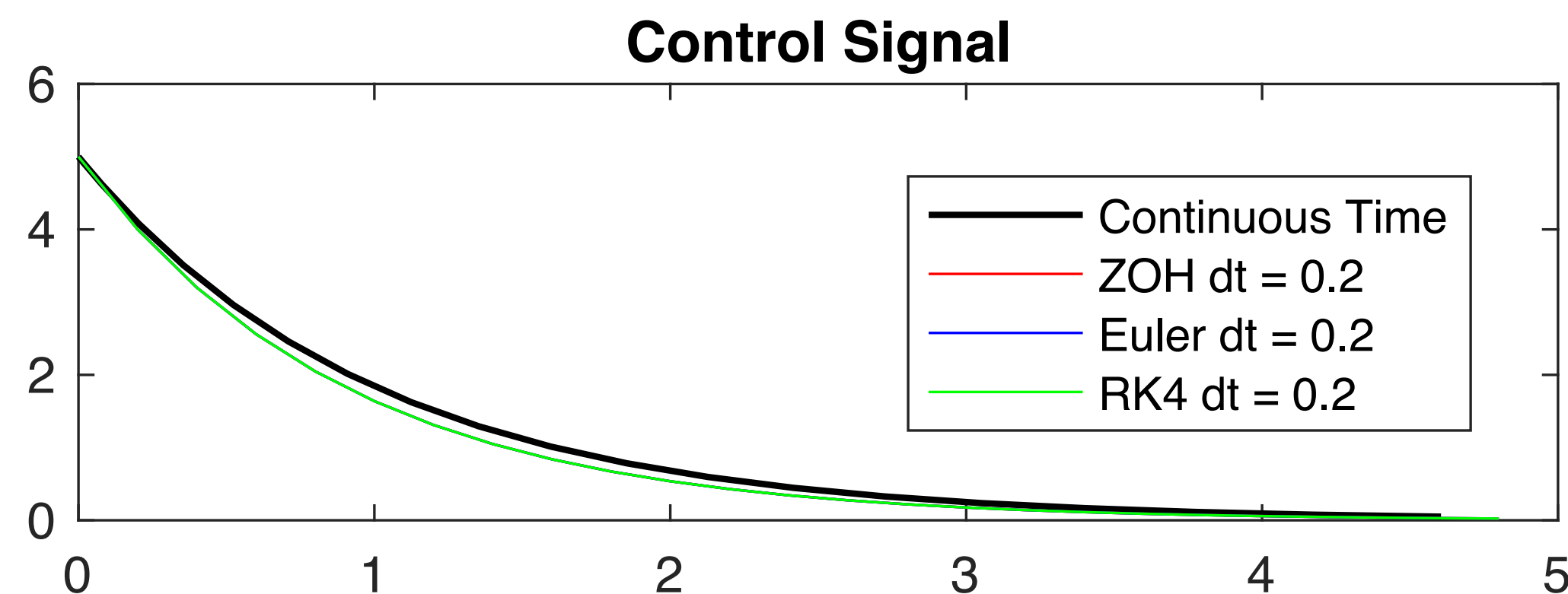
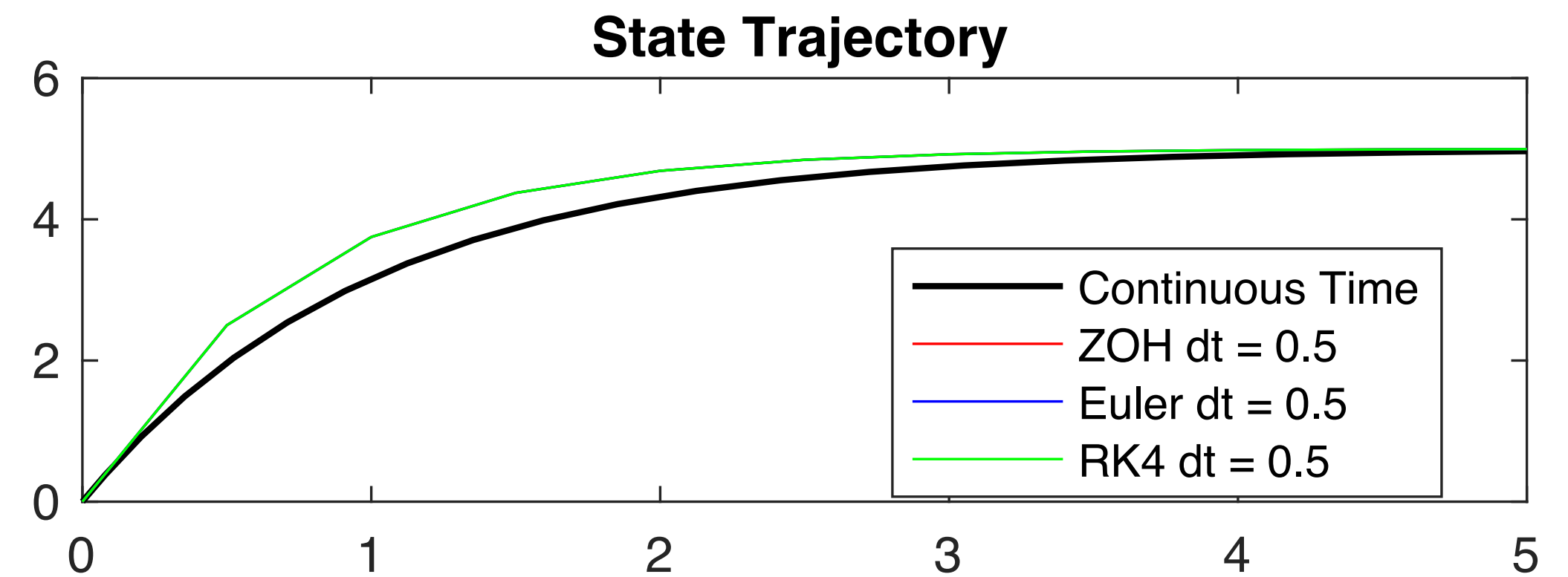
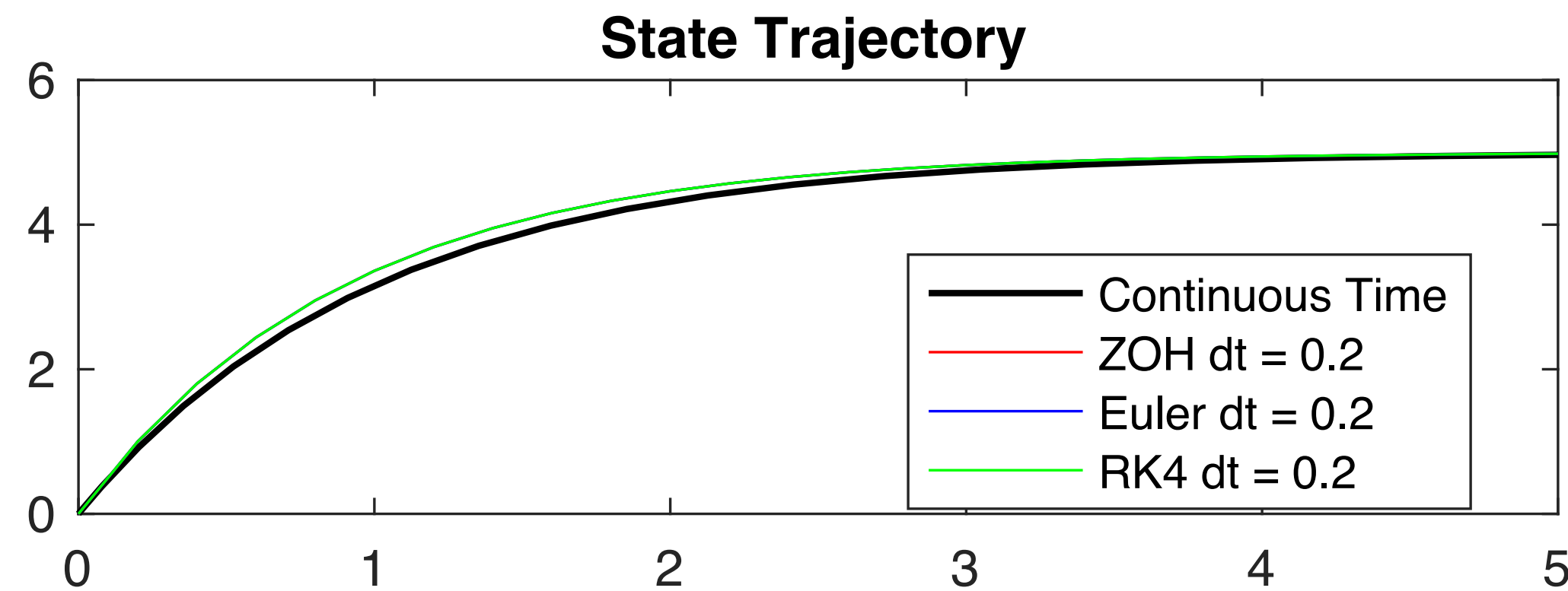
- RK4



<https://lpsa.swarthmore.edu/NumInt/NumIntFourth.html>

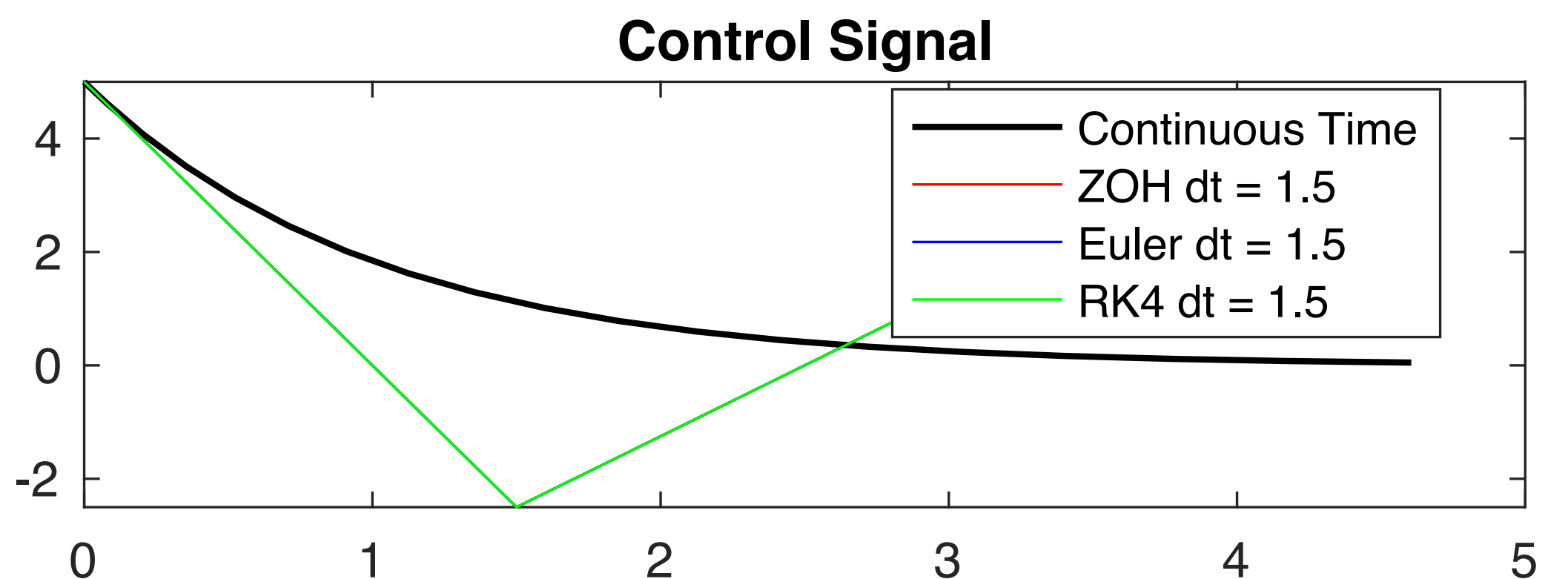
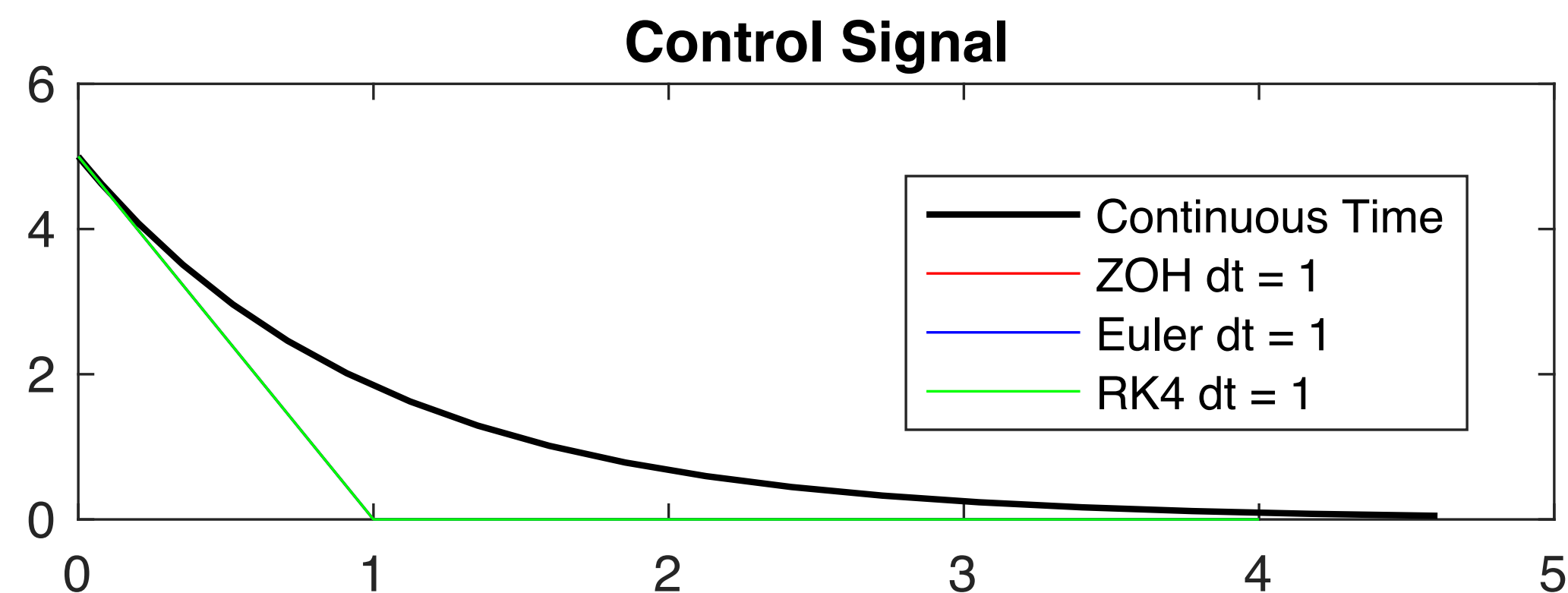
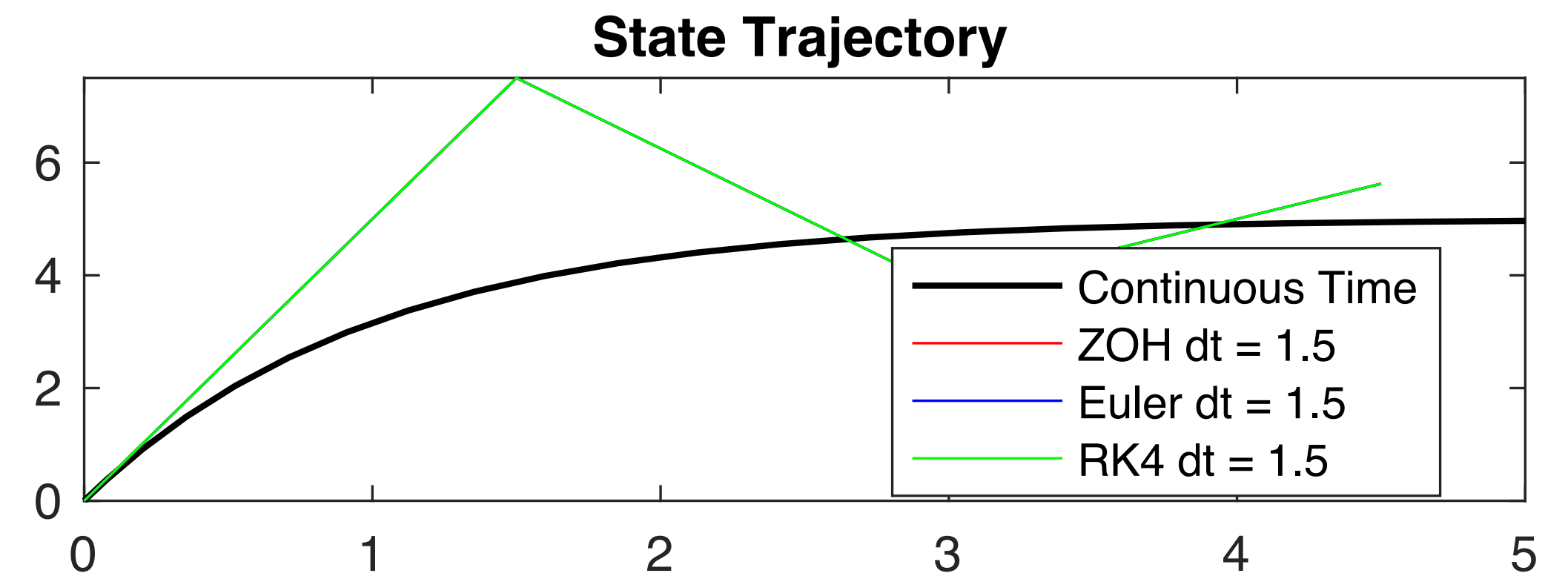
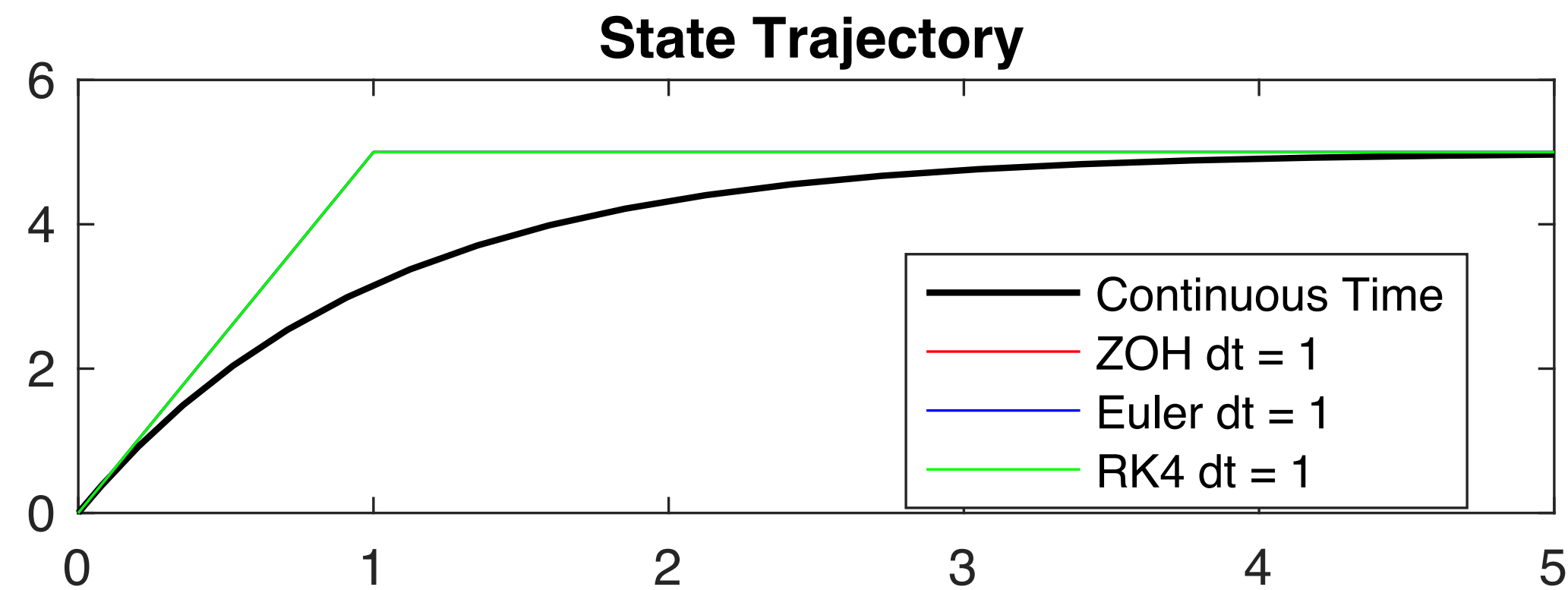
Mismatch Between CT & DT

* There is no difference among the three discretization methods



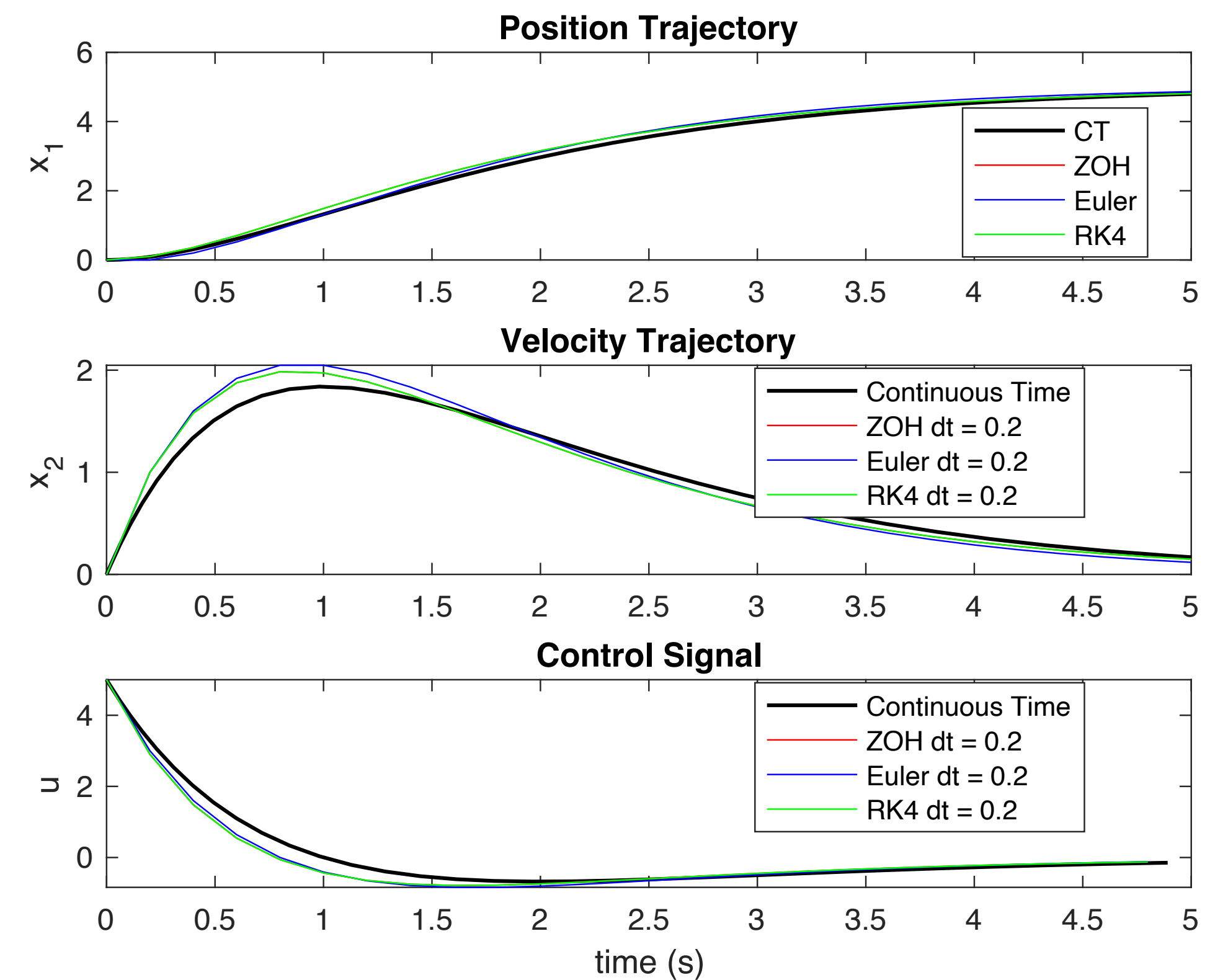
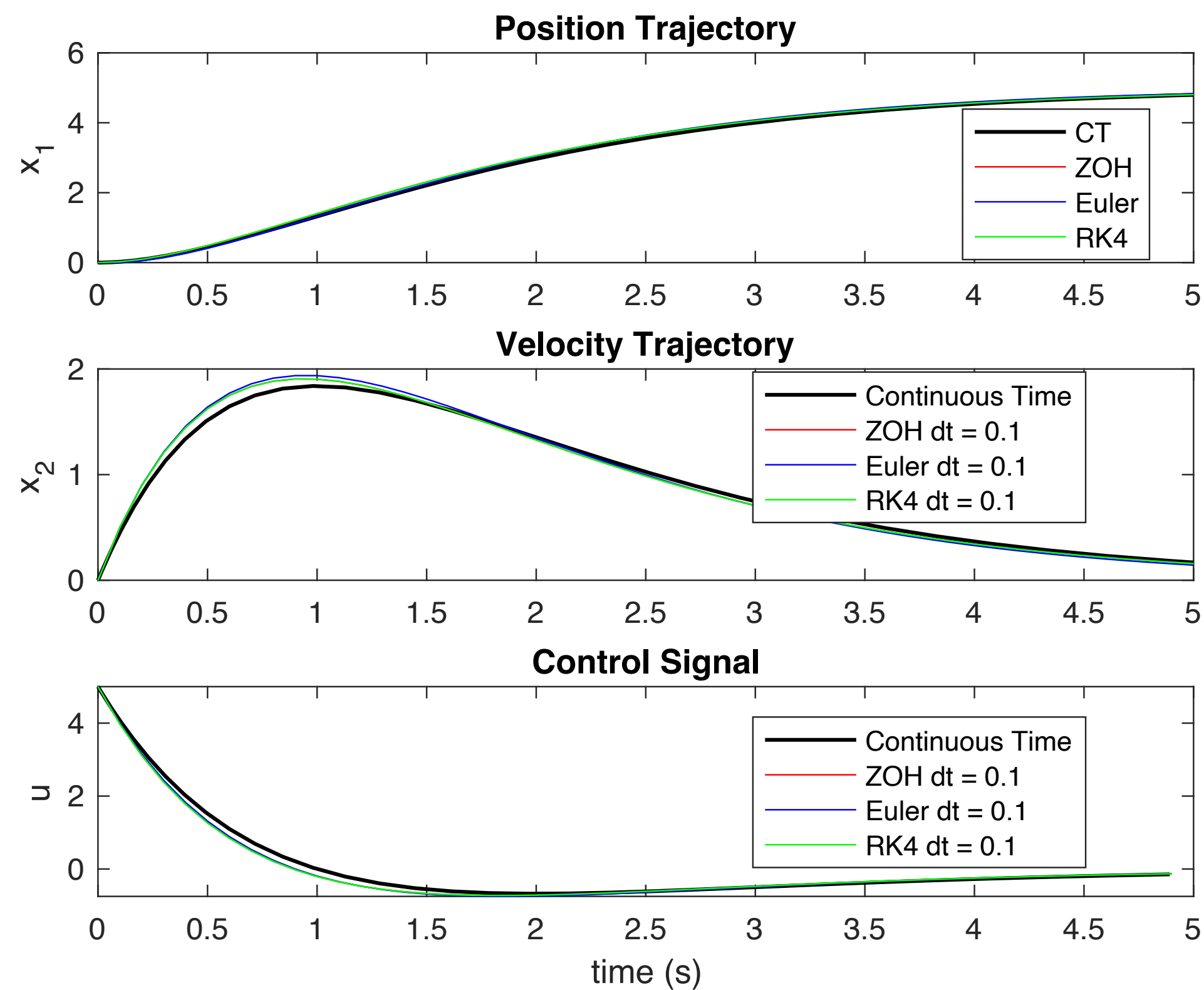
```
roll_out("single_integrator", @(x,t) 5 - x, 0, simCT);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_ZOH);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_Euler);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_RK4);
```


Mismatch Between CT & DT



```
roll_out("single_integrator", @(x,t) 5 - x, 0, simCT);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_ZOH);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_Euler);
roll_out("single_integrator", @(x,t) 5 - x, 0, simDT_RK4);
```

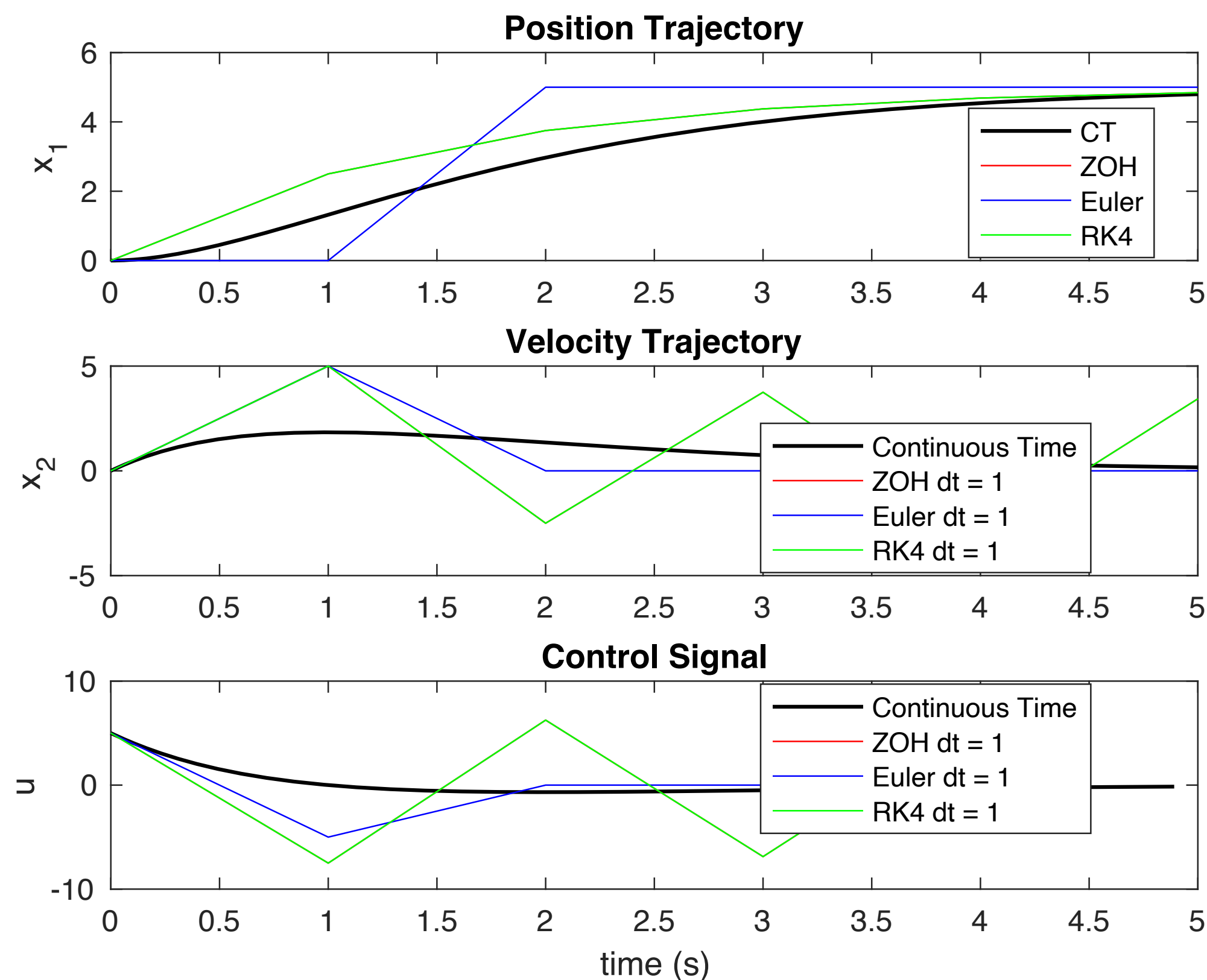
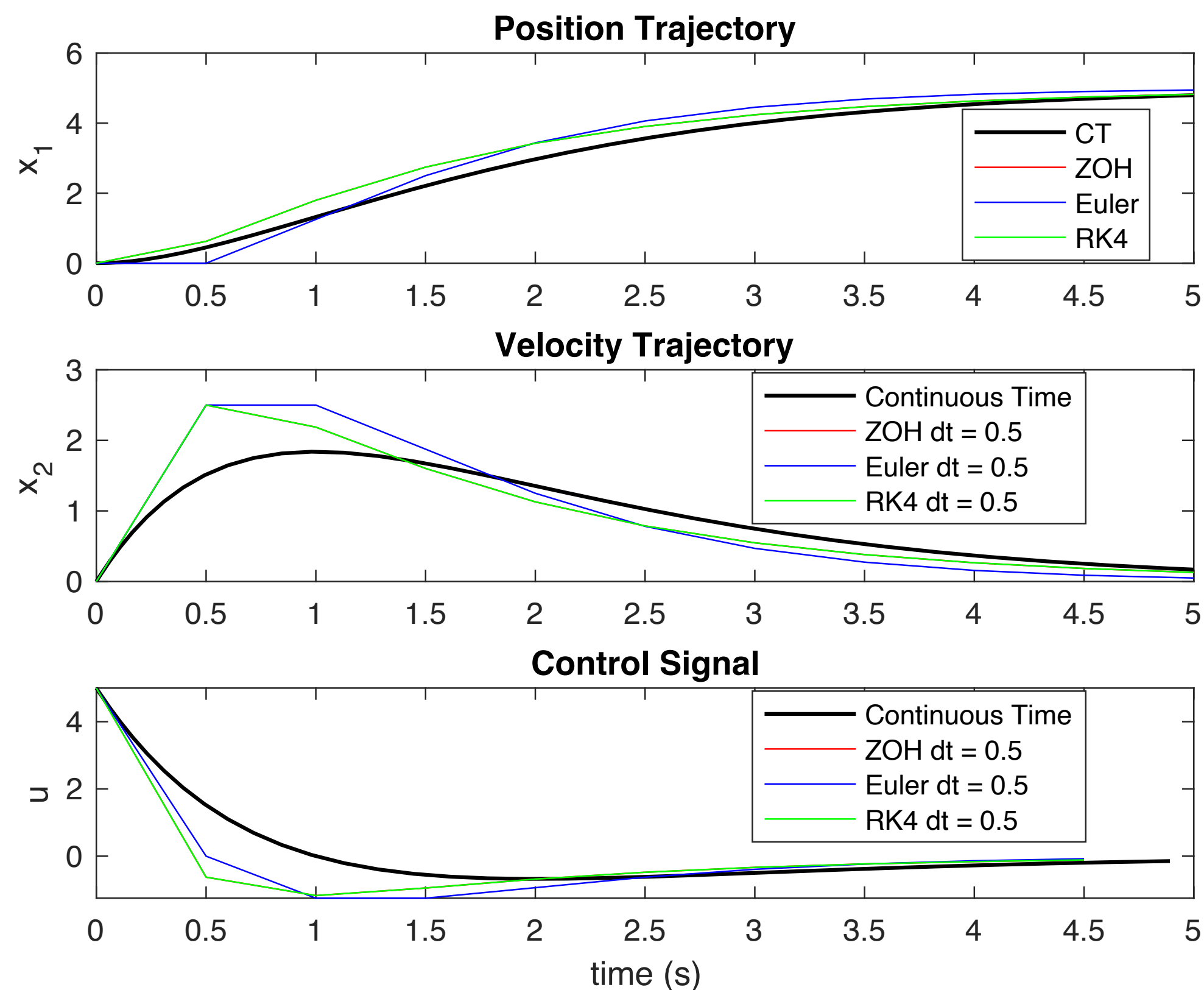
Mismatch Between CT & DT



```
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simCT);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_ZOH);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_Euler);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_RK4);
```

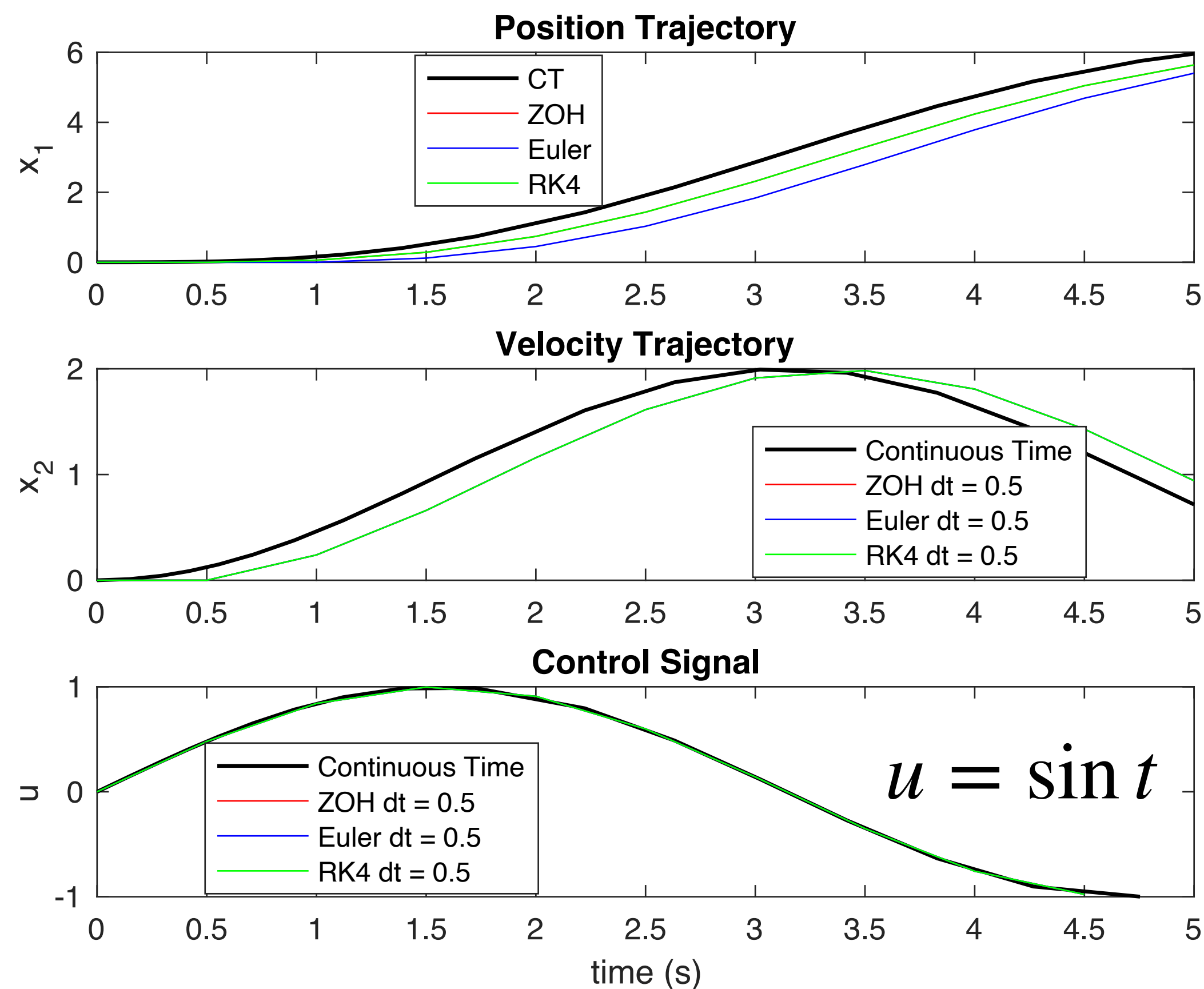
* State feedback control

Mismatch Between CT & DT

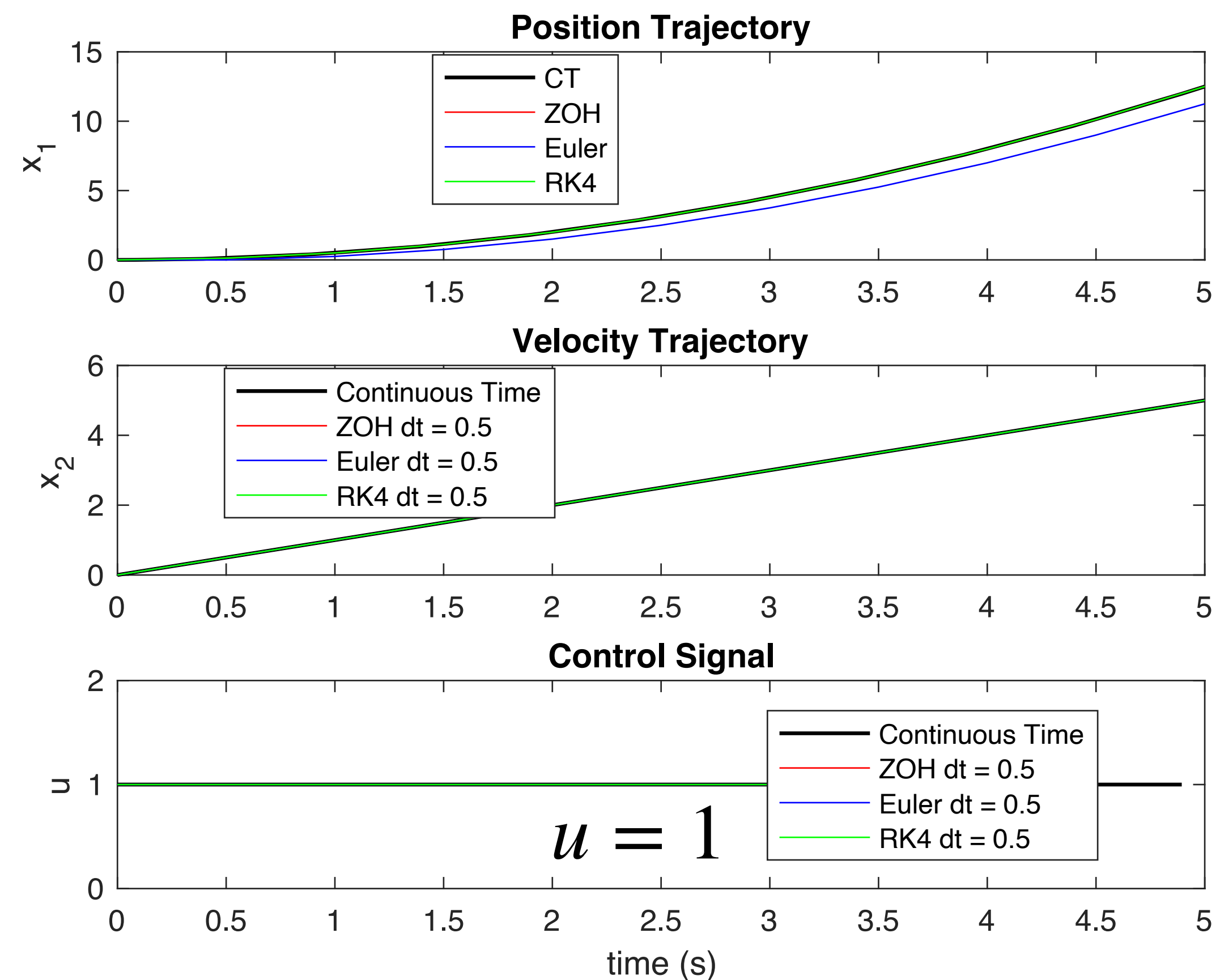


```
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simCT);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_ZOH);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_Euler);
roll_out("double_integrator", @(x,t) 5 - x(1) - 2*x(2), [0;0], simDT_RK4);
```

Mismatch Between CT & DT



```
roll_out("double_integrator", @(x,t) sin(t), [0;0], simCT);
roll_out("double_integrator", @(x,t) sin(t), [0;0], simDT_ZOH);
roll_out("double_integrator", @(x,t) sin(t), [0;0], simDT_Euler);
roll_out("double_integrator", @(x,t) sin(t), [0;0], simDT_RK4);
```

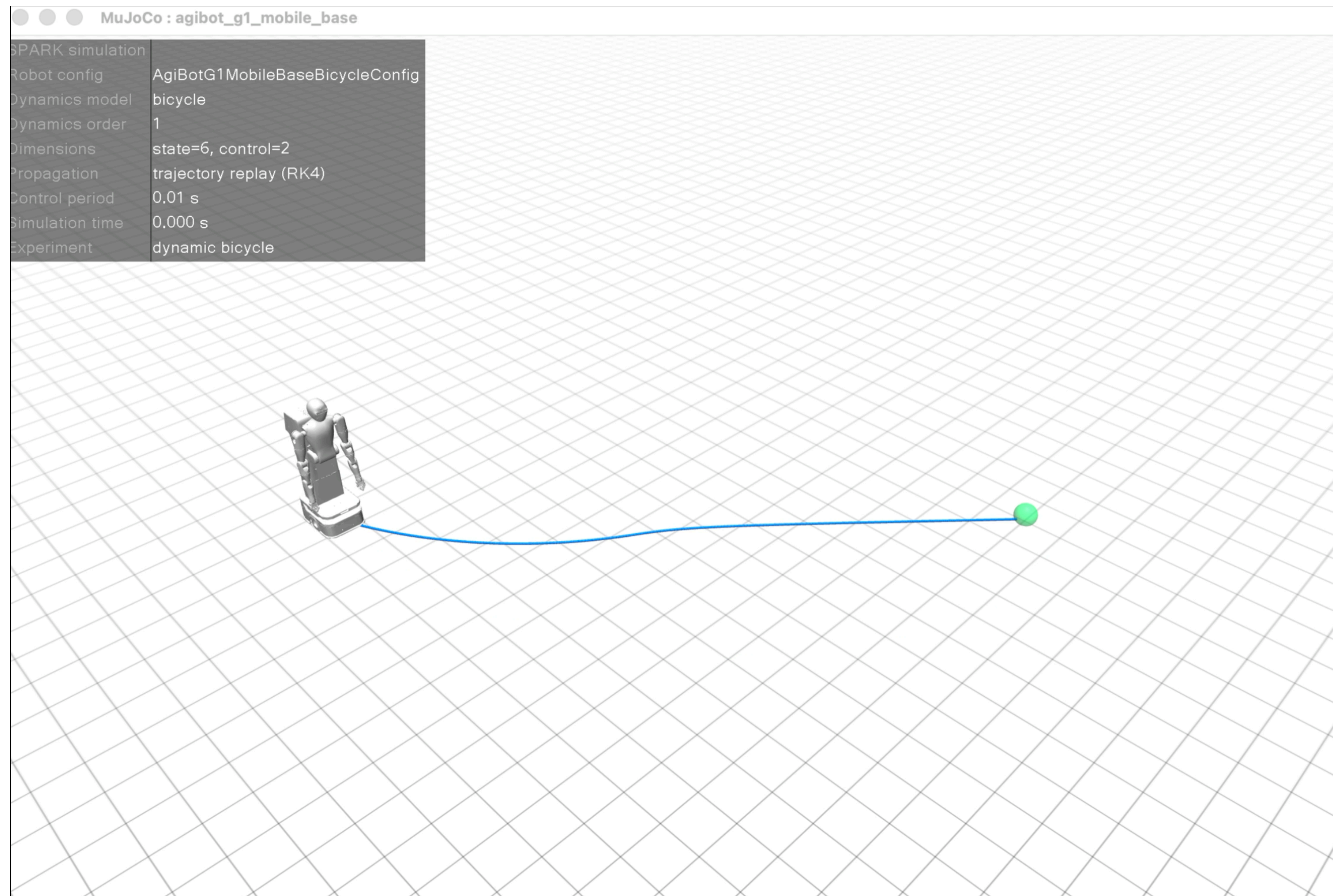


```
roll_out("double_integrator", @(x,t) 1, [0;0], simCT);
roll_out("double_integrator", @(x,t) 1, [0;0], simDT_ZOH);
roll_out("double_integrator", @(x,t) 1, [0;0], simDT_Euler);
roll_out("double_integrator", @(x,t) 1, [0;0], simDT_RK4);
```

Remark

- Under the same control law, the gaps between CT trajectories and DT trajectories could become bigger with larger sampling time, especially when the control signal is state-dependent or time-dependent
- In practice, people design different controllers for DT systems to account for the sampling time.

Vehicle Trajectory Visualization



Next Time

- Models for Manipulators